FORMAT FOR PREPARATION OF PROJECT REPORT

FOR

Foundations of Artificial Intelligence

LOGICLEAN: AN AUTONOMOUS CLEANING ROBOT SIMULATION USING CLASSICAL ARTIFICIAL INTELLIGENCE

A PROJECT REPORT

Submitted by

2116230701216 – JHASMITHA D

2116230701258 – KEERTHI PRIYA T



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "LOGIClean: An Autonomous Cleaning Robot Simulation Using Classical Artificial Intelligence" is the bonafide work of "…JHASMITHA D and KEERTHI PRIYA T…" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
1	INTRODUCTION	1
2	LITERATURE REVIEW	2
3	PROPOSED SYSTEM	4
	3.1 ARCHITECTURE DIAGRAM	4
4	MODULES DESCRIPTION	7
	4.1 ENVIRONMENT MODEL AND SENSING MODULE	7
	4.2 SEARCH AND EXPLORATION MODULE	7
	4.3 LOGICAL REASONING MODULE	9
	4.4 CONSTRAINT SATISFACTION MODULE	9
	4.5 COVERAGE PLANNING AND CLEANING CONTROLLER MODULE	10
	4.6 DYNAMIC OBSTACLE AVOIDANCE MODULE	11
5	IMPLEMENTATION AND RESULTS	13
	5.1 EXPERIMENTAL SETUP	13
	5.2 RESULTS	14
6	CONCLUSION AND FUTURE WORK	16
7	REFERENCES	17

ABSTRACT

LOGIClean is a desktop simulation of an autonomous floor-cleaning robot built entirely on classical, non-learning artificial intelligence. The robot is modelled as an intelligent agent that has no prior knowledge of the house: it senses only its four neighbouring cells, builds a belief map by frontier-based exploration, and plans every move with weighted A* search. A first-order-logic forward-chaining rule engine decides which room to skip, for example the room in which a pet is sleeping, and a constraint satisfaction problem solved with AC-3 and backtracking fixes the order in which rooms are visited. Cleaning is a complete boustrophedon coverage sweep performed one room at a time, so the robot never needs to know where the dirt is, and moving obstacles and the pet are handled by trajectory prediction and replanning within a finite battery. On seven different floor plans, mapping took 484 to 1,132 moves and a full cleaning session 826 to 1,280 moves; every dirty cell was cleaned, and no room was revisited.

CHAPTER 1
INTRODUCTION

Background

A robot vacuum has to solve, in one inexpensive appliance, several classical problems of artificial intelligence: it must build a representation of a house it has never seen, plan collision-free routes, decide in which order the rooms are cleaned, obey household rules and react to people and pets that move (Prassler et al., 2000). Many commercial products hide these decisions inside learned models. For a course on the Foundations of Artificial Intelligence the opposite is more instructive: a system in which every decision comes from a named textbook algorithm that can be traced, tested and explained.

Problem statement

Design and implement a simulated cleaning robot that, given only a floor plan it has never seen, (i) discovers the plan through limited sensing, (ii) cleans every room completely, one room at a time, (iii) obeys household rules, (iv) avoids moving obstacles and a pet, and (v) works within a finite battery, using only classical, non-learning AI techniques.

Objectives

- Model the robot as an intelligent agent with a belief state kept separate from the true world.
- Use informed search (A*) and frontier-based exploration to plan and to map an unknown space.
- Represent household knowledge in first-order logic and apply it by forward chaining.
- Formulate room ordering as a constraint satisfaction problem solved with AC-3 and backtracking.
- Predict and avoid moving hazards by trajectory extrapolation and replanning.

Scope

The world is a 40 × 30 grid of 20 cm cells. The robot moves one cell per tick and senses only its four neighbouring cells. The project deliberately contains no machine learning, and calendar-style scheduling was left out because it is a product feature rather than an AI technique. The user interface (map editor, Explore and Simulate views) is only a means of observing the AI and is not described in detail.

CHAPTER 2
LITERATURE REVIEW

Search and path planning

Hart, Nilsson and Raphael (1968) introduced A*, which expands nodes in order of f(n) = g(n) + h(n), where g is the cost so far and h a heuristic estimate of the remaining cost. If h never overestimates, the first path found is optimal (Russell and Norvig, 2021). On a four-connected grid the Manhattan distance is such a heuristic, even when some cells cost more to cross.

Mapping and exploration

Elfes (1989) proposed occupancy grids, which record for each cell whether it is occupied. Yamauchi (1997) introduced frontier-based exploration: a frontier is a known free cell next to unknown space, and repeatedly travelling to the nearest frontier maps the whole reachable environment. Thrun, Burgard and Fox (2005) add probabilistic sensor models, which LOGIClean replaces by a deterministic three-valued map (unknown, free, occupied).

Coverage path planning

Coverage planning asks for a path that passes over every point of an area (Choset, 2001; Galceran and Carreras, 2013). The boustrophedon decomposition of Choset and Pignon (1998) splits the free space into cells, sweeps each with back-and-forth motion and visits the cells in an order given by their adjacency. LOGIClean uses the rooms of the house as the cells.

Constraint satisfaction

A constraint satisfaction problem has variables, finite domains and constraints (Dechter, 2003). Mackworth (1977) introduced arc consistency and the AC-3 algorithm, which prunes domain values that cannot be part of any solution; backtracking search with the minimum-remaining-values heuristic then assigns the variables (Russell and Norvig, 2021).

Logical knowledge representation

Kowalski (1979) presented logic as a language for problem solving. Forward chaining applies definite-clause rules to known facts until no new conclusion follows (Russell and

Norvig, 2021). LOGIClean stores household state as ground facts and applies rules such as “pet sleeping in room R implies skip R”.

Dynamic obstacle avoidance

For a robot moving one cell at a time, a simple scheme is sufficient: predict where each moving hazard will be a few steps ahead and replan around those cells (LaValle, 2006).

Intelligent agents and task environments

Russell and Norvig (2021) define an agent as anything that perceives its environment through sensors and acts on it through actuators, and they describe a task environment by its performance measure, environment, actuators and sensors (PEAS). Environments are further classified as fully or partially observable, deterministic or stochastic, static or dynamic, and discrete or continuous. A goal-based agent chooses actions by considering their consequences for a goal, which requires search or planning. A robot vacuum is a typical goal-based agent in a partially observable, dynamic environment, and this classification drives the design choices of Chapter 3.

Cleaning robots

Prassler et al. (2000) reviewed the development of cleaning robots and the navigation problems that they pose. The central difficulty is that useful cleaning requires the whole floor to be covered while the machine carries only inexpensive, short-range sensors and starts without a map. A simple machine can react to its bumpers, whereas a more capable one builds a map and plans its routes over it. LOGIClean follows the second approach in a simplified form: a range-one sensor builds the map, and every later movement is planned on that map.

Partial observability and belief

When an environment is only partially observable the agent has to keep an internal representation of what it has learned so far. Thrun, Burgard and Fox (2005) call this the belief and update it after every action and measurement. LOGIClean uses the simplest possible belief: a grid in which each cell is unknown, free or occupied, updated deterministically from the four sensor readings. Because the sensor and the movement are exact, unless the optional drift model is switched on, the belief never contradicts the true map, and planning on the belief is the same as planning on everything that has been seen so far.

CHAPTER 3
PROPOSED SYSTEM

LOGIClean treats the cleaning robot as a goal-based intelligent agent. Its task environment, in the PEAS form of Russell and Norvig (2021), is:

- **Performance measure:** dirty cells cleaned, each room finished before the next begins, cells travelled, no collisions, battery never exhausted.
- **Environment:** a 40 × 30 grid house with walls, rooms, carpet, no-go zones, moving obstacles and a pet.
- **Actuators:** move one cell north, south, east or west; clean the current cell; dock and charge.
- **Sensors:** four-neighbour obstacle sensor, dirt and carpet sensor, odometry and battery level.

The environment is partially observable (the robot sees only adjacent cells), sequential, dynamic (obstacles and the pet move) and discrete. Partial observability is the central design constraint: the robot keeps its own belief map and plans only on what it has sensed.

3.1 ARCHITECTURE DIAGRAM

Figure 3.1 shows the system as five layers. Dependencies point downwards only, so the AI reasoning layer can be run and tested without the presentation layer.

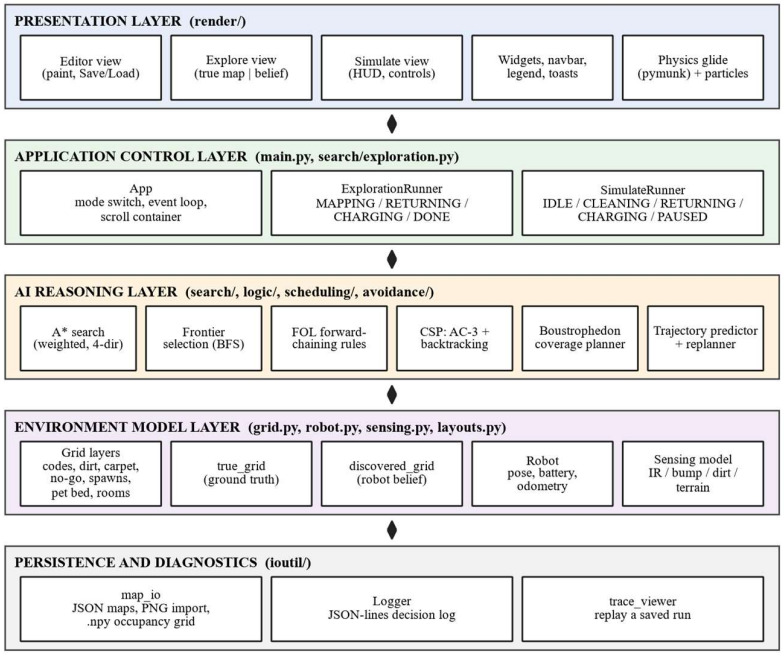


Figure 3.1: Layered architecture of LOGIClean

The ground-truth grid is read by a single function, sense(), which returns the four neighbouring cells with their dirt and terrain. The result is folded into the discovered grid, which starts entirely UNKNOWN, and every planner reads only that belief. The Simulate controller is a finite state machine with the phases IDLE, CLEANING, RETURNING, CHARGING and PAUSED: it sweeps the rooms in the order fixed by the constraint solver, returns to the dock when finished, stuck or stopped, and recharges and resumes when the battery is low.

Figure 3.2 shows one cycle of the perception–planning loop that every runner repeats once per tick. The robot reads its four neighbouring cells with sense(), folds the reading into the discovered grid and plans on that grid alone. Only then does it move, which changes the true state and begins the next cycle. Because the true map is read in a single function, the boundary between belief and truth can be checked by inspection and tested directly.

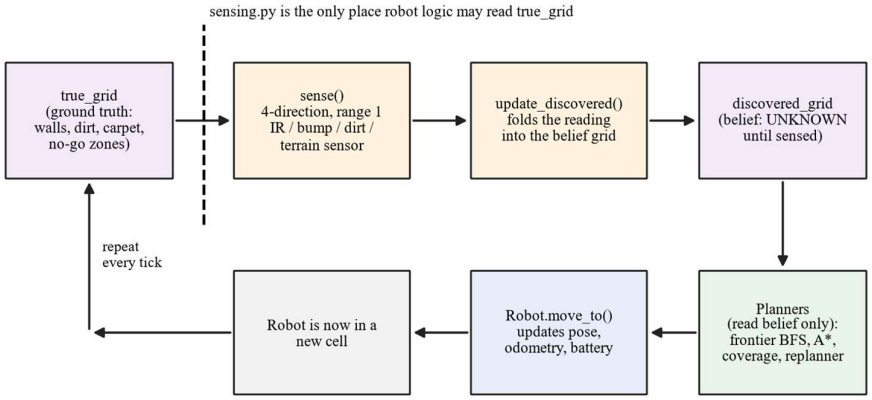


Figure 3.2: Perception–planning loop and the belief/truth boundary

Prior knowledge

The robot is given three pieces of knowledge before it starts: the position of its dock, the rectangles that label the rooms of the house, and the household rules, such as skipping a room in which the pet is sleeping. Everything else, the walls, the carpet, the no-go zones and the dirt, has to be sensed. The room labels are treated as authored metadata, in the way a user would name the rooms in a robot vacuum's app, and they are copied onto the belief grid so that the cleaning controller never reads the true map.

Design principles

- **Belief is separate from truth:** robot logic works on a discovered grid that starts entirely unknown, and the ground-truth grid is read only inside sense().
- **Every decision is explainable:** each behaviour comes from a named algorithm, and decisions are written to a JSON-lines log that can be replayed.
- **The AI is independent of the display:** the planners and controllers run without a window, which is how the automated tests and the experiments of Chapter 5 were carried out.
- **Failure is bounded:** stuck cells, unreachable cells and low battery are handled by explicit rules, so a session always terminates.

CHAPTER 4
MODULES DESCRIPTION

4.1 ENVIRONMENT MODEL AND SENSING MODULE

The world is a Grid whose aspects are held in parallel layers: cell codes (OPEN, WALL, DIRT, UNKNOWN), dirt intensity, carpet, no-go zones, obstacle spawns, the pet bed and the authored room rectangles. Two grids exist at run time: true_grid, the ground truth, and discovered_grid, the robot's belief. The Robot stores pose, odometer and battery (100 %, 0.03 % per cell, recharge below 25 %). sense() looks at the four neighbours and update_discovered() writes the reading into the belief, so sensing is the only place where robot logic touches the true map. Room labels are authored metadata, like the dock, and are copied onto the belief grid.

For each of the directions N, S, E and W, sense() returns a tuple of the cell code, the dirt intensity, whether the cell is carpet and whether it is a no-go zone. A wall or a no-go cell is therefore discovered the first time the robot stands next to it, without driving into it, and carpet or dirt is known one cell before the robot reaches it. update_discovered() writes the four readings into the belief grid and marks the robot's own cell open. An optional odometry model makes a step land one cell off course, perpendicular to the direction of travel, with a configurable probability; the movement code never accepts a landing cell that is not walkable, and after a drifted step the rest of the path is discarded and planned again.

4.2 SEARCH AND EXPLORATION MODULE

All routes are planned with A* and the Manhattan heuristic on the four-connected grid. Entering a carpet cell costs 2.0 instead of 1.0, which keeps the heuristic admissible, and an optional blocked set lets callers route around temporary hazards.

```
function ASTAR(grid, start, goal, blocked):
    open <- priority queue by f = g + h;  g[start] <- 0
    while open is not empty:
        cur <- cell with lowest f
        if cur = goal: return path rebuilt through came_from
        for each walkable 4-neighbour n of cur not in blocked:
            tentative <- g[cur] + cell_cost(n)   # 1.0, or 2.0 on carpet
            if tentative < g[n]:
                came_from[n] <- cur;  g[n] <- tentative
                push n with f = tentative + manhattan(n, goal)
    return None
```

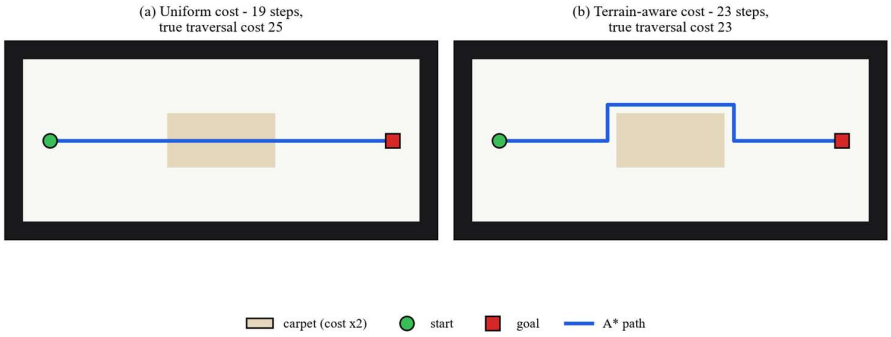


Figure 4.1: A* with uniform cost and with terrain cost: ignoring the rug costs 25 units, the terrain-aware plan 23

Every cell costs at least one unit, so the Manhattan distance never overestimates, the heuristic is admissible, and the first path that reaches the goal is a cheapest one. During exploration the search treats UNKNOWN cells as blocked, so the robot travels only over cells that it has already seen to be open. If the goal cannot be reached the search returns no path and the caller treats the goal as unreachable. Because the grid has at most 1,200 cells, planning is cheap; the decision cost per tick measured in Chapter 5 is a fraction of a millisecond.

Exploration is frontier-based. select_frontier() runs a breadth-first search outward from the robot over known walkable cells and returns the nearest cell with an UNKNOWN neighbour; if there is none, the map is complete. ExplorationRunner repeats sense, update belief, choose frontier, plan with A* over known cells and step. When no frontier remains the robot returns to its dock (Figure 4.2).

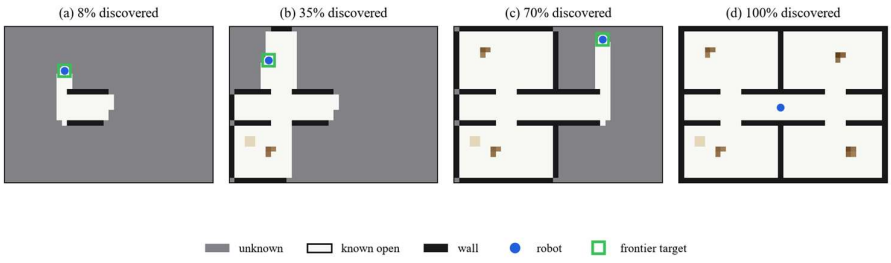


Figure 4.2: Belief map during exploration of the default house

Exploration ends when no frontier remains, which happens exactly when every reachable open cell has been sensed. A cell that is still UNKNOWN at that moment can never be seen by a range-one sensor, because all of its neighbours are walls, as at an outer corner; it is inferred to be a wall so that the boundary of the map is closed. The robot then plans a route to its dock over the belief and stops there. If the battery falls to the low threshold

during mapping, the same return-and-charge behaviour is used, and mapping resumes from the nearest frontier after the recharge.

4.3 LOGICAL REASONING MODULE

Household state is stored as ground facts, tuples such as ("pet_sleeping", "room_3"), in a FactSet. A Rule has a list of condition predicates and one conclusion predicate; the RuleEngine applies all rules by forward chaining every tick. The shipped rule is pet_sleeping(R) ⇒ skip(R). The pet's state is synchronised into the fact set each tick, so the cleaning controller skips the room that contains the pet's bed exactly while the pet is asleep (Figure 4.3).

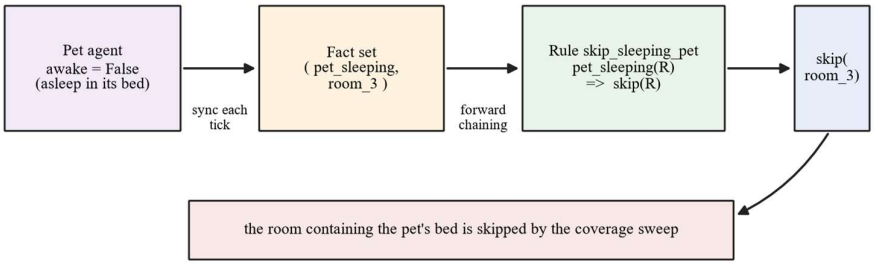


Figure 4.3: Forward chaining from the pet's state to a skipped room

The rule is evaluated every tick in four steps. First, the controller asserts the fact pet_sleeping(room) while the pet is asleep and retracts it when the pet wakes. Second, the RuleEngine finds every argument for which all condition predicates of a rule hold and produces the conclusion skip(room). Third, when the controller takes the next room from the CSP order it looks the room up in the set of skipped rooms, logs "skipping room (rule active)" and moves on to the following room. Fourth, when the pet wakes, the fact is retracted, the conclusion disappears at the next evaluation and the room becomes eligible again. A rule with several conditions fires only for an argument that satisfies all of them, so further household rules can be added as data without changing the controller.

4.4 CONSTRAINT SATISFACTION MODULE

The room order is posed as a CSP. The variables are the rooms and the values are visiting slots 0, 1, 2, …. The constraints are all-different, kitchen before bedroom, and study not first (the last two apply when those rooms exist). CSPScheduler enforces arc consistency with AC-3, then runs backtracking search that extends the variable with the fewest remaining values.

```
function BACKTRACK(assignment):
    if every room is assigned: return assignment
    room <- unassigned room with fewest remaining slots   # MRV
    for slot in domain[room] in increasing order:
        if slot is unused and constraints still hold:
            assignment[room] <- slot
            result <- BACKTRACK(assignment)
            if result is not failure: return result
            remove room from assignment
    return failure
```

For the default house the solver returns the order kitchen, bedroom, study, corridor, living room (Figure 4.4).

The pruning is visible in the domains of the rooms. The time window removes slot 0 from the study. For the constraint kitchen before bedroom, AC-3 removes the last slot from the kitchen, because no later slot exists for the bedroom, and the first slot from the bedroom, because no earlier slot exists for the kitchen. Backtracking then assigns the rooms in order of fewest remaining values and finds a consistent assignment at the first attempt, without a single dead end. The all-different constraint is checked during the search, so a slot that another room already holds is never assigned twice. If no assignment existed, the rooms would keep the order in which the map lists them, so that a cleaning session can always start.

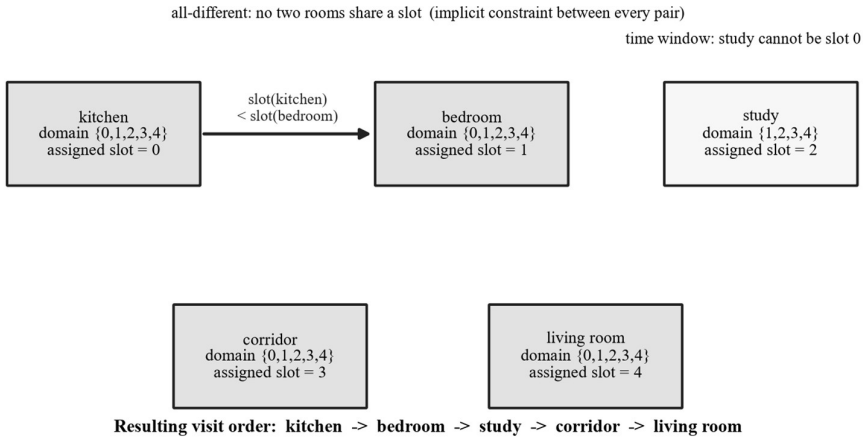


Figure 4.4: Constraint network and solution for the default house

4.5 COVERAGE PLANNING AND CLEANING CONTROLLER MODULE

A real vacuum cannot know where the dirt is, so the robot does not path to dirty cells. generate_room_sweep() lists every walkable cell of a room row by row, reversing the direction on alternate rows; consecutive waypoints are joined by A*, and any dirt is cleaned because the robot stops on every cell. The controller holds the waypoints of one

room only and takes the next room from the CSP order when, and only when, that queue is empty, so a room is always finished before the next begins (Figure 4.5).



Figure 4.5: One cleaning session, coloured by room and numbered in visiting order

A waypoint with no progress for 15 ticks is skipped, and three skipped waypoints in a row send the robot home. At 25 % battery the waypoint in progress is re-queued and cleaned after the recharge; a cell unreachable on a full charge is dropped after two attempts, so a session always terminates.

Dirt is modelled as an intensity between 0 and 1 on each dirty cell. Every time the robot stands on a dirty cell the intensity falls by 0.4, so a heavily soiled cell may need more than one pass. After a cooldown of 10 ticks a cleaned site regrows by 0.02 per tick, up to its original intensity, which makes repeated sessions meaningful. Carpet costs twice as much to cross, so the paths that join waypoints prefer bare floor, and no-go cells are neither swept nor entered. A session passes through the phases IDLE, CLEANING, RETURNING and CHARGING and can be paused and resumed; pausing freezes the robot while the pet and the obstacles continue to move.

4.6 DYNAMIC OBSTACLE AVOIDANCE MODULE

Obstacles patrol along an axis from their spawn cell. The pet cycles between sleeping in its bed (40 ticks) and wandering (25 ticks). The predictor extrapolates the direction of a hazard's latest step for three cells; the replanner blocks the hazard's cell and the predicted cells, and runs A* again only if they meet the next three cells of the robot's plan. The blocks are recomputed every tick, so a cell is released as soon as it is no longer predicted to be occupied (Figure 4.6).

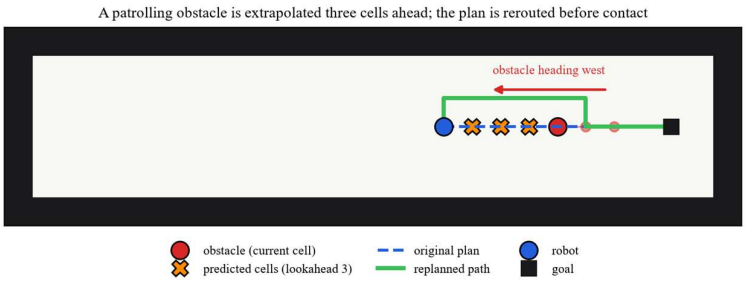


Figure 4.6: Prediction of a moving obstacle and the rerouted path

Each obstacle keeps the last eight positions it has occupied, and the predictor uses the final two of them. By default an obstacle follows a deterministic patrol, because a linear prediction is only meaningful for a predictable trajectory; a random-walk mode exists as a stress test. The pet exposes the same position and history interface, so it is handled by exactly the same prediction and replanning code. The user can also take manual control of the pet, sending it to a clicked cell or nudging it one cell with the arrow keys, and then hand it back to its autonomous routine. Every replanning event is written to the decision log together with the blocked cells and whether a new path was found.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The system is written in Python 3.11 with pygame (display), NumPy (grids) and pytest. The experiments ran headlessly through the same Explore and Simulate code paths as the interface, in the order Explore then Simulate with the same Robot, on seven floor plans (Figure 5.1): the default house and six plans that differ in structure. For each map the number of cells travelled, recharge trips, cleaned dirty cells, room order and room revisits were recorded. Runs were capped at 60,000 ticks so that a hang would be reported as a failure.

Cells moved is taken from the robot's odometer, recharge trips count the returns to the dock forced by the battery, and the room sequence records the order of the rooms so that any revisit can be detected. The headless runs include the patrolling obstacles of each map but not the pet, whose rule and sleep cycle are covered by the unit tests.

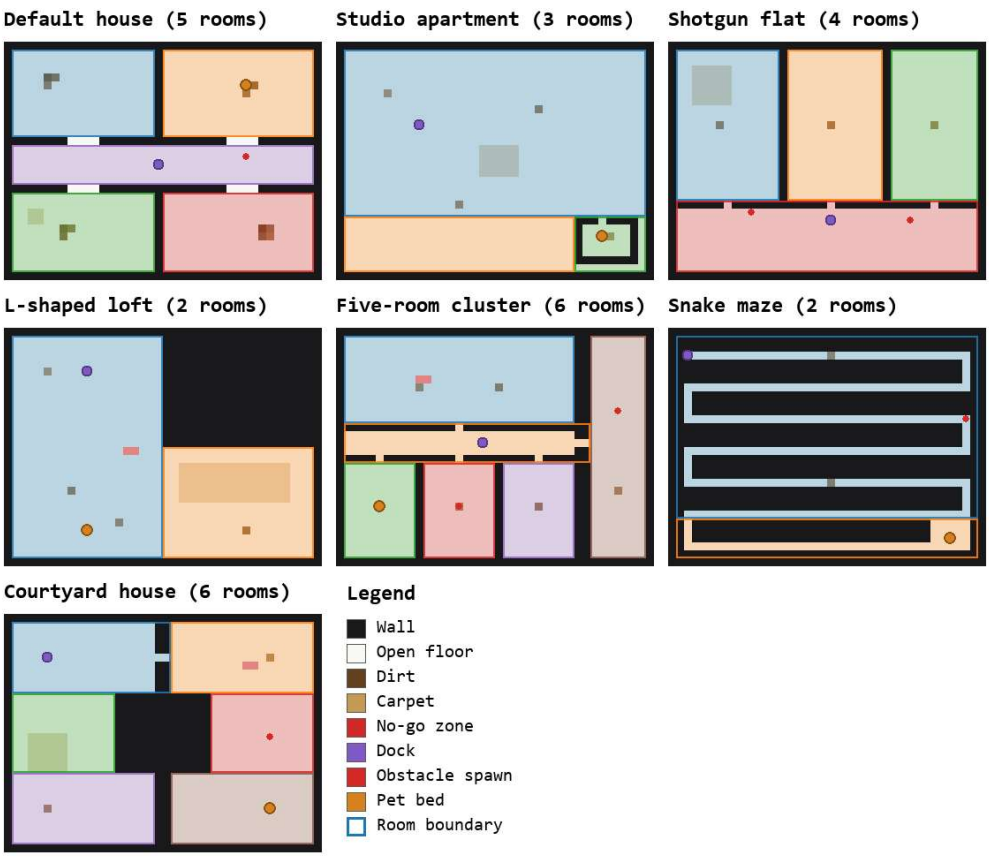


Figure 5.1: The seven test floor plans with their authored rooms

5.2 RESULTS

For each map, the cells moved while mapping, the cells moved during one full cleaning session and the dirty cells cleaned were:

- **Default house:** 1,132 and 1,194 cells, 13 of 13 dirty cells cleaned.
- **Studio apartment:** 1,076 and 1,280 cells, 4 of 4 dirty cells cleaned.
- **Shotgun flat:** 1,110 and 1,212 cells, 3 of 3 dirty cells cleaned.
- **L-shaped loft:** 824 and 896 cells, 4 of 4 dirty cells cleaned.
- **Five-room cluster:** 1,106 and 1,180 cells, 6 of 6 dirty cells cleaned.
- **Snake maze:** 484 and 826 cells, 3 of 3 dirty cells cleaned.
- **Courtyard house:** 952 and 1,188 cells, 4 of 4 dirty cells cleaned.

On all seven maps mapping left no reachable cell unknown, no room was visited twice, the robot ended at its dock and 0 recharge trips were needed. Mapping took 484 to 1,132 cell moves and a cleaning session 826 to 1,280. A session is much longer than the number of dirty cells suggests; this is the price of not knowing where the dirt is.

Dividing the cells moved by the number of walkable cells shows how efficient each phase is. Mapping needed between 1.03 and 1.19 cells per walkable cell on the six open plans and 1.70 on the snake maze; cleaning needed 1.13 to 1.31 and 2.90. On the open plans the serpentine sweep is therefore close to one visit per cell. The maze is the hard case: consecutive rows of the sweep lie in different arms of the winding corridor and must be joined by long detours.

The battery drain per cell changed the result strongly. The original 1.6 % per cell, calibrated for a coarser grid, gives a range of only 47 cells; mapping finished on 4 of 7 maps, with 28.8 recharges on average, and on the other three the robot shuttled between the dock and a frontier it could not reach on one charge until the 60,000-tick cap stopped it. At 0.2 % (375 cells per charge) mapping finished on 7 of 7 maps with 1.7 recharges on average, and at the final 0.03 % (2,500 cells per charge) on 7 of 7 with none. Exploration has no guard against the livelock, so the drain was recalibrated.

The room model mattered as much. On the shotgun flat, rectangles hard-coded for the default house gave 3,050 cells moved and generic quadrants 2,802, against 1,212 with authored rooms, because a real room split across two zones was swept in two distant visits. The decision logic itself is cheap, 0.03 to 0.11 ms per tick.

AI techniques

- **A*:** with an admissible heuristic every route is optimal, and carpet cost changed the chosen route exactly where the rug lay (Figure 4.1).
- **Frontier exploration:** it is complete for the reachable space; on every map no reachable cell remained unknown, and unreachable corners were correctly inferred to be walls.
- **Constraint satisfaction:** AC-3 pruned the domains before search, so the five-room problem was solved without any backtracking; the order always respected the constraints.
- **Forward chaining:** the skip rule fired exactly while the pet slept, so the pet's room was never entered during that time.
- **Coverage and replanning:** complete sweeping cleaned every dirty cell without knowing where it was, and replanning kept the robot clear of the predicted cells of moving hazards.

Verification

The behaviour described above is checked by automated tests rather than by inspection. Besides the interface and input/output tests, the suite contains:

- 18 tests of A*, frontier selection and exploration (routing around walls and no-go zones, unreachable goals, carpet avoidance).
- 24 tests of the rule engine (firing and retraction), the CSP solver (required order, time windows, unsatisfiable problems) and the serpentine coverage sweep.
- 21 tests of obstacle prediction, replanning and the pet's sleep and wake cycle.
- 20 tests of the cleaning controller, including the recharge re-queue and its bounded retries.
- 14 end-to-end tests that run all seven maps through Explore and Simulate and check that mapping finishes, the session ends at the dock and no room is revisited.

CHAPTER 6
CONCLUSION AND FUTURE WORK

Conclusion

LOGIClean shows that a complete household-robot behaviour can be built from classical AI alone. The robot maps an unknown house with frontier exploration and A*, orders the rooms with a constraint satisfaction solver, applies household rules by forward chaining, cleans each room completely with a boustrophedon sweep and avoids moving hazards by prediction and replanning, all within a finite battery. Belief and truth are kept apart and every decision is logged, so the behaviour can be explained. On seven different floor plans every session finished, every dirty cell was cleaned, no room was revisited, and all 173 automated tests pass.

Limitations

- Rooms are authored rather than inferred; a map without rooms is swept as one room.
- The sensor is noise-free with range one, and obstacle prediction assumes a constant heading.
- A session sweeps every cell once, so heavy dirt is reduced by 0.4 per pass, not removed.

Future work

- Infer rooms automatically by boustrophedon cellular decomposition (Choset and Pignon, 1998).
- Order rooms by travel distance as well as constraints, reusing the simulated-annealing module (Kirkpatrick, Gelatt and Vecchi, 1983).
- Add a battery-aware frontier choice with a livelock guard.
- Introduce noisy sensing and a probabilistic occupancy grid (Thrun, Burgard and Fox, 2005).

CHAPTER 7
REFERENCES

- Choset, H. (2001) 'Coverage for robotics – A survey of recent results', Annals of Mathematics and Artificial Intelligence, Vol.31, No.1–4, pp.113-126.
- Choset, H. and Pignon, P. (1998) 'Coverage path planning: The boustrophedon cellular decomposition', in Zelinsky, A. (ed.) Field and Service Robotics, London: Springer, pp.203-209.
- Dechter, R. (2003) Constraint Processing, San Francisco: Morgan Kaufmann.
- Elfes, A. (1989) 'Using occupancy grids for mobile robot perception and navigation', Computer, Vol.22, No.6, pp.46-57.
- Galceran, E. and Carreras, M. (2013) 'A survey on coverage path planning for robotics', Robotics and Autonomous Systems, Vol.61, No.12, pp.1258-1276.
- Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, Vol.4, No.2, pp.100-107.
- Kirkpatrick, S., Gelatt, C.D. and Vecchi, M.P. (1983) 'Optimization by simulated annealing', Science, Vol.220, No.4598, pp.671-680.
- Kowalski, R. (1979) Logic for Problem Solving, New York: North-Holland.
- LaValle, S.M. (2006) Planning Algorithms, Cambridge: Cambridge University Press.
- Mackworth, A.K. (1977) 'Consistency in networks of relations', Artificial Intelligence, Vol.8, No.1, pp.99-118.
- Prassler, E., Ritter, A., Schaeffer, C. and Fiorini, P. (2000) 'A short history of cleaning robots', Autonomous Robots, Vol.9, No.3, pp.211-226.
- Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edn, Hoboken: Pearson.
- Thrun, S., Burgard, W. and Fox, D. (2005) Probabilistic Robotics, Cambridge, MA: MIT Press.
- Yamauchi, B. (1997) 'A frontier-based approach for autonomous exploration', Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA), Monterey, CA, pp.146-151.